

Lab Assignment 1

JDBC CRUD Operations using Statement

1. Title

Implementation of CRUD Operations using JDBC Statement

2. Aim

To develop Java programs that perform **Create, Read, Update, and Delete (CRUD)** operations on a MySQL database using the Statement interface.

3. Learning Objectives

- Establish database connectivity using JDBC
- Use Statement to execute SQL queries
- Perform INSERT, SELECT, UPDATE, DELETE operations
- Understand the limitations of Statement
- Handle SQL exceptions

4. Software Requirements

- JDK 8 or above
- Eclipse IDE
- MySQL Server
- MySQL Connector/J Driver
- Database: college

5. Database Setup

Step 1: Create Database

```
CREATE DATABASE exp;  
USE college;
```

Step 2: Create Table

```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course VARCHAR(30),  
    marks INT  
);
```

6. Theory Background

What is JDBC?

JDBC (Java Database Connectivity) is an API that enables Java programs to interact with relational databases.

What is a Statement?

Statement is an interface in the java.sql package used to execute static SQL queries.

Example:

```
Statement stmt = con.createStatement();
```

7. Assignment Questions

Problem statement 1.1.: Write a Java program using Statement to insert a student record into the student table.

Requirements:

- Accept input from user (id, name, course, marks)
- Execute INSERT query
- Display number of rows inserted

Code:

```
package com.exp;
import java.sql.*;
public class InsertStudent {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/lab", "root", "Shriyash@11");

            Statement stmt = con.createStatement();

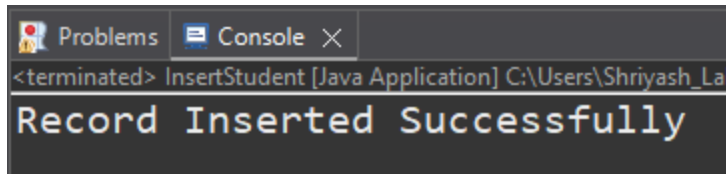
            String query = "INSERT INTO student VALUES (108, 'Neha', 'MCA', 85)";
            stmt.executeUpdate(query);

            System.out.println("Record Inserted Successfully");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

```
}  
}
```

Output:



DB:

```
mysql> SELECT * FROM STUDENT;  
+-----+-----+-----+-----+  
| id   | name    | course | marks |  
+-----+-----+-----+-----+  
| 199  | Shriyash | MCA    | 85    |  
+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

Problem statement 1.2. Write a program using Statement to retrieve a student record based on ID.

Requirements:

- Accept student ID
- Execute SELECT query
- Display record if found
- Display “Record Not Found” if not present

Code:

```
package com.exp;  
import java.sql.*;  
import java.util.Scanner;  
  
public class GetStudentById {  
    public static void main(String[] args) {  
        try {  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/lab", "root", "Shriyash@11");  
  
            Statement stmt = con.createStatement();  
            System.out.print("Enter Student ID:");  
            Scanner sc = new Scanner(System.in);  
  
            int id = sc.nextInt();
```

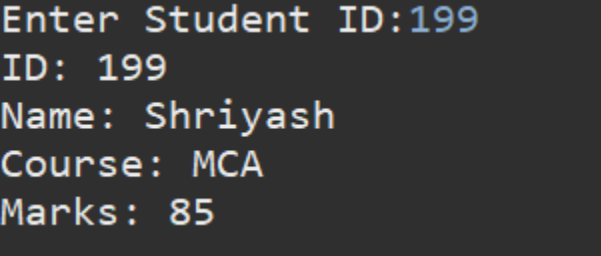
```
String query = "SELECT * FROM student WHERE id = " + id;

ResultSet rs = stmt.executeQuery(query);

if (rs.next()) {
    System.out.println("ID: " + rs.getInt("id"));
    System.out.println("Name: " + rs.getString("name"));
    System.out.println("Course: " + rs.getString("course"));
    System.out.println("Marks: " + rs.getInt("marks"));
} else {
    System.out.println("Record not found");
}

con.close();
} catch (Exception e) {
    System.out.println(e);
}
}
```

Output



```
Enter Student ID:199
ID: 199
Name: Shriyash
Course: MCA
Marks: 85
```

Problem statement 1.3. Write a program to display all records from the student table.

Requirements:

- Use executeQuery()
- Display records in tabular format
- Use ResultSet

Code:

```
package com.exp;
import java.sql.*;

public class DisplayAllStudents {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
```

```
"jdbc:mysql://localhost:3306/lab", "root", "Shriyash@11");
```

```
Statement stmt = con.createStatement();
```

```
String query = "SELECT * FROM student";
```

```
ResultSet rs = stmt.executeQuery(query);
```

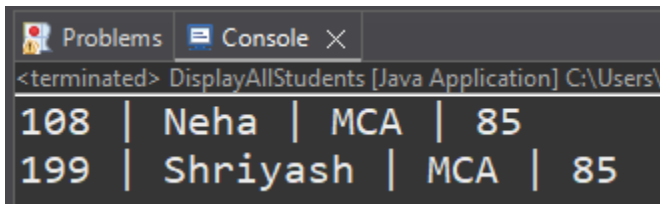
```
while (rs.next()) {  
    System.out.println(  
        rs.getInt("id") + " | " +  
        rs.getString("name") + " | " +  
        rs.getString("course") + " | " +  
        rs.getInt("marks")  
    );  
}
```

```
con.close();
```

```
} catch (Exception e) {  
    System.out.println(e);  
}
```

```
}  
}
```

Output



```
<terminated> DisplayAllStudents [Java Application] C:\Users\  
108 | Neha | MCA | 85  
199 | Shriyash | MCA | 85
```

Problem statement 1.4. Write a Java program using Statement to update marks of a student.

Requirements:

- Accept ID and new marks
- Execute UPDATE query
- Display number of rows updated

Code:

```
package com.exp;

import java.sql.*;

public class UpdateStudent {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/lab", "root", "Shriyash@11");

            Statement stmt = con.createStatement();

            int id = 199;
            int newMarks = 95;

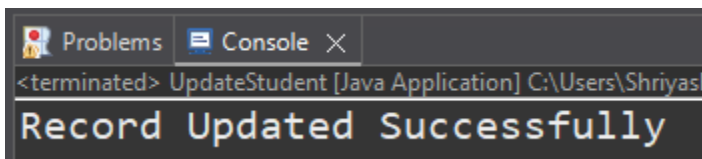
            String query = "UPDATE student SET marks = " + newMarks + " WHERE id = " + id;

            int rows = stmt.executeUpdate(query);

            if (rows > 0) {
                System.out.println("Record Updated Successfully");
            } else {
                System.out.println("Record Not Found");
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:



DB:

```
mysql> SELECT * FROM STUDENT;
```

id	name	course	marks
108	Neha	MCA	85
199	Shriyash	MCA	95

```
2 rows in set (0.00 sec)
```

Problem statement 1.5. Write a Java program using Statement to delete a record.

Requirements:

- Accept student ID
- Execute DELETE query
- Display number of rows deleted

Code:

```
package com.exp;

import java.sql.*;

public class DeleteStudent {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/lab", "root", "Shriyash@11");

            Statement stmt = con.createStatement();

            int id = 199;
            String query = "DELETE FROM student WHERE id = " + id;

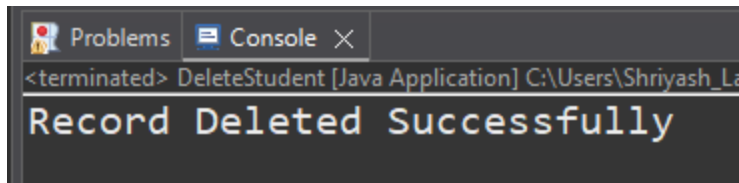
            int rows = stmt.executeUpdate(query);

            if (rows > 0) {
                System.out.println("Record Deleted Successfully");
            } else {
                System.out.println("Record Not Found");
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

```
}
```

Output:



DB:

```
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+
| id  | name | course | marks |
+-----+-----+-----+-----+
| 108 | Neha | MCA    | 95    |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Problem statement 1.6.: Write a **menu-driven Java program** using Statement to perform all CRUD operations.

Menu Structure

1. Insert Student
2. Search Student
3. Display All Students
4. Update Student Marks
5. Delete Student
6. Exit

Functional Requirements

- Establish database connection once
- Use Statement for all operations
- Display appropriate messages
- Handle invalid input
- Close connection properly

Code

Web Development with Java – MCA Sem II

```
import java.sql.*;
import java.util.Scanner;

public class StudentCRUD {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "Mysql@1234"
            );

            Statement stmt = con.createStatement();

            int choice;

            do {
                System.out.println("\n---- MENU ----");
                System.out.println("1. Insert");
                System.out.println("2. Search");
                System.out.println("3. Display All");
                System.out.println("4. Update");
                System.out.println("5. Delete");
                System.out.println("6. Exit");

                choice = sc.nextInt();

                switch(choice) {

                    case 1:
                        System.out.println("Enter id, name, course, marks:");
                        int id = sc.nextInt();
                        String name = sc.next();
                        String course = sc.next();
                        int marks = sc.nextInt();

                        stmt.executeUpdate(
                            "INSERT INTO student VALUES(" +
                                id + "," + name + "," +
                                course + "," + marks + ")"
                        );
                        System.out.println("Record Inserted");
                        break;

                    case 2:
```

```
System.out.println("Enter id:");
int sid = sc.nextInt();

ResultSet rs = stmt.executeQuery(
    "SELECT * FROM student WHERE id=" + sid
);

if(rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name") + " " +
        rs.getString("course") + " " +
        rs.getInt("marks")
    );
} else {
    System.out.println("Record Not Found");
}
break;
```

case 3:

```
ResultSet rs2 = stmt.executeQuery(
    "SELECT * FROM student"
);

while(rs2.next()) {
    System.out.println(
        rs2.getInt("id") + " " +
        rs2.getString("name") + " " +
        rs2.getString("course") + " " +
        rs2.getInt("marks")
    );
}
break;
```

case 4:

```
System.out.println("Enter id and new marks:");
int uid = sc.nextInt();
int newMarks = sc.nextInt();

int rows = stmt.executeUpdate(
    "UPDATE student SET marks=" +
    newMarks + " WHERE id=" + uid
);

System.out.println(rows + " record updated");
break;
```

case 5:

```
System.out.println("Enter id:");
int did = sc.nextInt();
```

```
        int del = stmt.executeUpdate(
            "DELETE FROM student WHERE id=" + did
        );

        System.out.println(del + " record deleted");
        break;

    case 6:
        System.out.println("Exiting...");
        break;

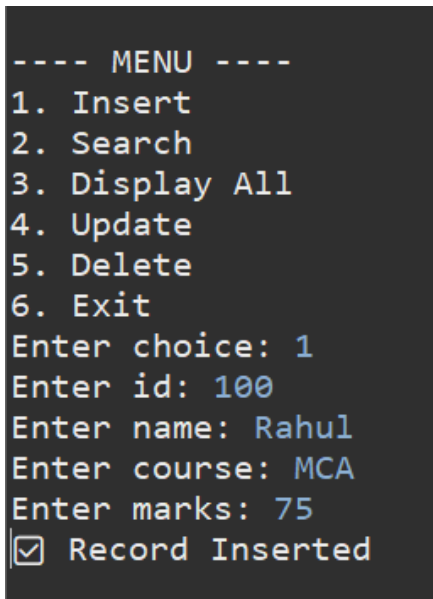
    default:
        System.out.println("Invalid Choice");
    }

} while(choice != 6);

con.close();
stmt.close();
sc.close();

} catch(Exception e) {
    System.out.println(e);
}
}
```

Output



```
---- MENU ----
1. Insert
2. Search
3. Display All
4. Update
5. Delete
6. Exit
Enter choice: 1
Enter id: 100
Enter name: Rahul
Enter course: MCA
Enter marks: 75
☑ Record Inserted
```

8. Conclusion

In this lab, CRUD operations are implemented using the Statement interface and the process of executing SQL queries through JDBC is understood.